

TaskApp Academica

Paso a paso del desarrollo de una aplicacion movil hibrida para gestionar tareas academicas con Ionic, Angular, dashboard visual, filtros y persistencia local.

1. Que pide la practica

La practica pide construir una aplicacion movil donde un estudiante pueda ver, crear, marcar como completadas y eliminar tareas academicas. No pide login, servidor, API ni base de datos externa. Por eso el proyecto se resolvio como frontend Ionic/Angular con persistencia local.

- Crear una pagina principal con componentes Ionic: ion-header, ion-title, ion-content e ion-footer.
- Mostrar tareas en una lista movil usando ion-list, ion-item e ion-label.
- Mostrar titulo, descripcion corta y registrar prioridad en el formulario.
- Usar un boton flotante ion-fab para abrir el formulario de nueva tarea.
- Definir una interfaz TypeScript llamada Tarea.
- Crear el formulario con ReactiveFormsModule y validaciones.
- Permitir marcar tareas como completadas con ion-checkbox y texto tachado.
- Permitir eliminar tareas con icono de papelera o deslizamiento.
- Como extra, centralizar la logica en un servicio Angular y guardar en localStorage.
- Mejorar la experiencia visual con resumen, barra de progreso y filtros por estado.

2. Aclaracion: Ionic, Angular y TaskApp

Ionic no es una base de datos. Ionic es un framework de componentes visuales para crear aplicaciones moviles hibridas con aspecto de app movil. Da componentes como barras, listas, botones flotantes, checkboxes y badges.

Angular es el framework que controla la logica de la aplicacion: componentes, formularios, validaciones, servicios, rutas y estado de los datos.

TaskApp es el nombre de la aplicacion que se debe crear. En este proyecto se llama TaskApp Academica porque esta enfocada en tareas escolares o academicas.

3. Frontend, backend y base de datos

Este proyecto solo tiene frontend. El frontend es la aplicacion visual creada con Ionic y Angular. No hay backend porque el enunciado no solicita API, servidor, autenticacion ni administracion centralizada de datos.

- No hace falta MySQL, PostgreSQL, MongoDB ni Firebase.
- No hace falta crear backend o API.
- No existe un comando para encender backend porque no hay backend en esta practica.
- Si el profesor pidiera usuarios, login o sincronizacion entre celulares, ahi si haria falta backend y base de datos.

- Aqui basta con un servicio Angular que lea y escriba el arreglo de tareas en localStorage.

4. Estructura del proyecto

El proyecto se organizo con una estructura clara para separar configuracion, modelo de datos, servicio y pagina principal.

Archivo o carpeta	Funcion
package.json	Define dependencias y comandos como npm install, npm run start y npm run build.
angular.json	Configura el proyecto Angular usado por Ionic.
ionic.config.json	Indica a Ionic que el proyecto es de tipo Angular.
src/app/models/tarea.model.ts	Define la interfaz Tarea y el tipo de prioridad.
src/app/services/tareas.service.ts	Centraliza crear, eliminar, completar y guardar tareas en localStorage.
src/app/pages/tareas/tareas.page.ts	Contiene la logica de pantalla, signals, filtros y navegacion al registro.
src/app/pages/tareas/tareas.page.html	Contiene dashboard, metricas, filtros, lista, checkbox, contador y boton flotante.
src/app/pages/tareas/tareas.page.scss	Define el diseno movil limpio, tarjetas, colores, estados y responsive.
src/app/pages/nueva-tarea/	Contiene la pagina independiente con formulario reactivo para registrar tareas.
README.md	Explica los comandos para instalar y ejecutar el proyecto.

5. Modelo de datos

Se definio una interfaz TypeScript para asegurar que todas las tareas tengan la misma forma. Esto ayuda a evitar errores y cumple el requisito del modelo de datos.

```
export interface Tarea {  
  id: number;  
  titulo: string;  
  descripcion: string;  
  prioridad: 'Alta' | 'Media' | 'Baja';  
  completada: boolean;  
}
```

6. Servicio de tareas

El servicio TareasService es el lugar donde se maneja el arreglo de tareas. Tiene metodos para agregar, cambiar estado y eliminar. Ademas, guarda los cambios en localStorage.

- agregarTarea crea una nueva tarea con id, titulo, descripcion, prioridad y completada en falso.
- cambiarEstado actualiza si una tarea esta completada o pendiente.
- eliminarTarea quita una tarea de la lista.
- actualizarTareas guarda el arreglo en localStorage.
- cargarTareas lee localStorage y carga datos iniciales si no hay informacion guardada.

7. Interfaz principal

La pantalla principal se diseño como una app movil academica: cabecera limpia, hero con resumen del dia, barra de progreso, tarjetas de metricas, filtros por estado, lista de tareas y acceso a una pagina de registro.

- ion-header e ion-title: parte superior de la app.
- ion-content: area principal donde vive el contenido.
- ion-progress-bar: muestra avance de tareas completadas.
- Botones de filtro: permiten ver todas, pendientes y completadas.
- ion-list e ion-item: lista movil de tareas.
- ion-checkbox: cambia el estado completada.
- ion-badge: muestra el contador de tareas filtradas.
- ion-button con ion-icon trash: elimina una tarea.
- ion-item-sliding: permite eliminar deslizando el elemento.
- ion-fab: boton flotante para abrir el formulario.
- ion-footer: pie de la pantalla.

8. Formulario reactivo

El formulario esta en una pagina independiente llamada nueva-tarea. Se usa ReactiveFormsModule para controlar campos y validaciones desde TypeScript.

- titulo: obligatorio y minimo 5 caracteres.
- descripcion: obligatoria y limitada a 120 caracteres.
- prioridad: se guarda por defecto como Media para mantener el formulario simple.
- Si el formulario es invalido, se muestran mensajes de error y no se guarda la tarea.

9. Paso a paso de desarrollo

- 1 Crear la carpeta del proyecto y abrirla en Visual Studio Code.
- 2 Crear la configuracion base de Angular e Ionic: package.json, angular.json e ionic.config.json.
- 3 Instalar dependencias con npm install.
- 4 Crear el modelo Tarea en src/app/models/tarea.model.ts.
- 5 Crear el servicio TareasService para manejar los datos y localStorage.
- 6 Crear la pagina TareasPage con componentes standalone de Ionic.
- 7 Construir la plantilla HTML con dashboard, metricas, filtros, lista, pagina de registro, fab y footer.
- 8 Agregar estilos SCSS para una presentacion limpia, compacta y movil.
- 9 Probar la compilacion con npm run build.

10 Ejecutar en desarrollo con ionic serve o npm run start.

10. Comandos necesarios

Instalar dependencias:

```
npm install
```

Ejecutar en desarrollo:

```
npm run start
```

Ejecutar el frontend con nombre explicito:

```
npm run start:frontend
```

Backend:

No hay backend que encender en esta practica.

Tambien puede ejecutarse directamente si se usa Ionic CLI:

```
ionic serve
```

Generar build de produccion:

```
npm run build
```

Generar build del frontend con nombre explicito:

```
npm run build:frontend
```

11. Entrega recomendada

- Subir el proyecto a GitHub sin la carpeta node_modules.
- Verificar que .gitignore incluya node_modules, www, dist y .angular.
- Adjuntar README.md con instrucciones de instalacion y ejecucion.
- Adjuntar este PDF como paso a paso del desarrollo.

12. Checklist final

- La app tiene interfaz Ionic completa.
- La app tiene dashboard, metricas, filtros y barra de progreso.
- La app permite crear tareas.
- La app valida el titulo obligatorio y minimo de 5 caracteres.
- La app permite marcar tareas como completadas.
- La app tacha el texto de tareas completadas.
- La app permite eliminar tareas.
- La app guarda datos en localStorage.
- El proyecto compila con npm run build.

13. Como defender el proyecto

- Si preguntan por backend: explicar que no se implemento porque el enunciado no lo pide; la persistencia solicitada se resolvió con localStorage.
- Si preguntan por base de datos: explicar que localStorage funciona como almacenamiento local del navegador para esta practica.
- Si preguntan por Ionic: explicar que aporta los componentes moviles usados en la interfaz.
- Si preguntan por Angular: explicar que maneja la logica, formularios reactivos, validaciones, signals y servicios.
- Si preguntan por ejecucion: npm install instala dependencias y npm run start enciende el frontend.

Conclusion: el proyecto cumple lo pedido sin usar backend ni base de datos externa. La persistencia se resolvió con localStorage, y la interfaz se mejoro para que parezca una aplicacion movil academica terminada.